OPERATING SYSTEMS LAB

Experiment 2: Lab Environment Setup

CPU Scheduling | FCFS | SJF | Python

Name:	Najim Khan
Roll No.:	2401420003
Course:	B.Tech CSE (DS)
Semester:	V Sem
Subject:	Operating Systems Lab

Aim: To write a Python program that implements FCFS and non-preemptive SJF CPU scheduling on the same set of processes, validates all input, shows CPU idle intervals, and displays the execution sequence and start/end intervals of each algorithm.

1 Problem Description

CPU scheduling decides which process in the ready queue gets the CPU next. In this experiment two non-preemptive algorithms are compared on one common dataset.

FCFS (First Come First Served)

1. Sort the processes by arrival time (ties are broken by the order in which they were entered).
2. Run each process to completion in that order.
3. If the next process has not arrived yet, the CPU stays idle until its arrival time.

SJF (Shortest Job First, non-preemptive)

1. At the current time, collect all processes that have already arrived.
2. Choose the one with the smallest burst time and run it to completion.
3. If no process has arrived, show a CPU idle interval and jump to the next arrival time.
4. Repeat until all processes are finished.

Tie-breaking rule used: if two processes have the same burst time, the one with the **earlier arrival time** runs first. If the arrival time is also equal, the process **entered first** by the user is chosen.

Formulae

Term	Formula
Completion Time (CT)	Time at which the process finishes
Turnaround Time (TAT)	CT - Arrival Time
Waiting Time (WT)	TAT - Burst Time

How each requirement is handled

Requirement	Implementation in the program
Accept PID, AT, BT	read_processes() takes input from the user
Validate input	read_int() re-asks until valid; duplicate or empty PID rejected
Preserve original data	original list is never changed; deepcopy is given to each algorithm
FCFS	fcfs() sorts by (arrival, entry order)
Non-preemptive SJF	sjf() picks min burst among arrived processes
Tie-breaking	key = (burst, arrival, entry order)
CPU idle intervals	IDLE entry added to the timeline when nothing has arrived
Separate output	show() prints sequence and intervals for each algorithm
Same dataset	both algorithms use copies of the same original list

2 Program (scheduling.py)

scheduling.py (part 1 of 2)

```
# FCFS and Non-Preemptive SJF CPU Scheduling
import copy


def read_int(prompt, minimum, allow_equal=True):
    """Keep asking until a valid integer >= minimum is entered."""
    while True:
        text = input(prompt).strip()
        try:
            value = int(text)
        except ValueError:
            print("  Invalid input: please enter a whole number.")
            continue
        if value < minimum or (value == minimum and not allow_equal):
            print(f"  Invalid input: value must be {'>=' if allow_equal else '>'} {minimum}.")
            continue
        return value


def read_processes():
    n = read_int("Enter number of processes: ", 1)
    processes, used_ids = [], set()
    for i in range(n):
        print(f"--- Process {i + 1} ---")
        while True:
            pid = input("  Process ID   : ").strip().upper()
            if not pid:
                print("  Invalid input: Process ID cannot be empty.")
            elif pid in used_ids:
                print("  Invalid input: Process ID already used.")
            else:
                break
        used_ids.add(pid)
        at = read_int("  Arrival Time : ", 0)
        bt = read_int("  Burst Time   : ", 1)
        processes.append({"pid": pid, "at": at, "bt": bt, "order": i})
    return processes


def build_result(processes, timeline):
    """Compute CT, TAT and WT from the execution timeline."""
    finish = {pid: end for pid, start, end in timeline if pid != "IDLE"}
    rows = []
    for p in processes:
        ct = finish[p["pid"]]
        tat = ct - p["at"]
        rows.append((p["pid"], p["at"], p["bt"], ct, tat, tat - p["bt"]))
    return rows


def fcfs(processes):
    # Order by arrival time; tie -> the process entered first
    queue = sorted(processes, key=lambda p: (p["at"], p["order"]))
    timeline, time = [], 0
    for p in queue:
        if time < p["at"]:
            timeline.append(("IDLE", time, p["at"]))
            time = p["at"]
        timeline.append((p["pid"], time, time + p["bt"]))
        time += p["bt"]
    return timeline


def sjf(processes):
    # Non-preemptive: shortest burst among arrived processes.
    # Tie-break: smaller burst -> earlier arrival -> entered first
```

scheduling.py (part 2 of 2)

```
    remaining, timeline, time = list(processes), [], 0
    while remaining:
        ready = [p for p in remaining if p["at"] <= time]
        if not ready:
            next_at = min(p["at"] for p in remaining)
            timeline.append(("IDLE", time, next_at))
            time = next_at
            continue
        p = min(ready, key=lambda x: (x["bt"], x["at"], x["order"]))
        timeline.append((p["pid"], time, time + p["bt"]))
        time += p["bt"]
        remaining.remove(p)
    return timeline


def show(title, processes, timeline):
    print("\n" + "=" * 58)
    print(f"  {title}")
    print("=" * 58)
    print("Execution Sequence :", " -> ".join(t[0] for t in timeline))
    print("\nStart/End Intervals:")
    for pid, start, end in timeline:
        label = "CPU IDLE" if pid == "IDLE" else pid
        print(f"  {label:<9}: {start:>3} - {end:<3}")
    rows = build_result(processes, timeline)
    print("\n PID   AT   BT   CT   TAT   WT")
    print("-" * 33)
    for r in rows:
        print(f" {r[0]:<5}{r[1]:<5}{r[2]:<5}{r[3]:<5}{r[4]:<6}{r[5]}")
    n = len(rows)
    print("-" * 33)
    print(f"Average Turnaround Time : {sum(r[4] for r in rows) / n:.2f}")
    print(f"Average Waiting Time    : {sum(r[5] for r in rows) / n:.2f}")


def main():
    print("FCFS and SJF (Non-Preemptive) Scheduling\n")
    original = read_processes()
    # Each algorithm gets its own deep copy; original data is never changed
    data_fcfs = copy.deepcopy(original)
    data_sjf = copy.deepcopy(original)

    print("\nOriginal Process Data (same dataset for both algorithms)")
    print(" PID   AT   BT")
    for p in original:
        print(f" {p['pid']:<5}{p['at']:<5}{p['bt']}")

    show("FCFS SCHEDULING", data_fcfs, fcfs(data_fcfs))
    show("SJF (NON-PREEMPTIVE) SCHEDULING", data_sjf, sjf(data_sjf))


if __name__ == "__main__":
    main()
```

3 Execution and Output

Run 1: Invalid inputs, tie in burst time and CPU idle time

Ubuntu Terminal

```
cd ~/os_lab
python3 scheduling.py
```

In this run wrong values are entered on purpose (text instead of number, 0 processes, duplicate Process ID, negative arrival time, zero burst time) to show that the validation works. Process P5 arrives late at time 20, so the CPU stays idle.

Ubuntu Terminal

```
FCFS and SJF (Non-Preemptive) Scheduling

Enter number of processes: abc
  Invalid input: please enter a whole number.
Enter number of processes: 0
  Invalid input: value must be >= 1.
Enter number of processes: 5
--- Process 1 ---
  Process ID   : p1
  Arrival Time : 0
  Burst Time   : 4
--- Process 2 ---
  Process ID   : P1
  Invalid input: Process ID already used.
  Process ID   : P2
  Arrival Time : -1
  Invalid input: value must be >= 0.
  Arrival Time : 1
  Burst Time   : 0
  Invalid input: value must be >= 1.
  Burst Time   : 3
--- Process 3 ---
  Process ID   : P3
  Arrival Time : 2
  Burst Time   : 3
--- Process 4 ---
  Process ID   : P4
  Arrival Time : 3
  Burst Time   : one
  Invalid input: please enter a whole number.
  Burst Time   : 1
--- Process 5 ---
  Process ID   : P5
  Arrival Time : 20
  Burst Time   : 2
```

Ubuntu Terminal

```
Original Process Data (same dataset for both algorithms)
 PID   AT   BT
 P1   0    4
 P2   1    3
 P3   2    3
 P4   3    1
 P5   20   2

============================================================
  FCFS SCHEDULING
============================================================
Execution Sequence : P1 -> P2 -> P3 -> P4 -> IDLE -> P5

Start/End Intervals:
  P1       :   0 - 4
  P2       :   4 - 7
  P3       :   7 - 10
  P4       :  10 - 11
  CPU IDLE :  11 - 20
  P5       :  20 - 22

 PID   AT   BT   CT   TAT   WT
---------------------------------
 P1   0    4    4    4     0
 P2   1    3    7    6     3
 P3   2    3    10   8     5
 P4   3    1    11   8     7
 P5   20   2    22   2     0
---------------------------------
Average Turnaround Time : 5.60
Average Waiting Time    : 3.00

============================================================
  SJF (NON-PREEMPTIVE) SCHEDULING
============================================================
Execution Sequence : P1 -> P4 -> P2 -> P3 -> IDLE -> P5

Start/End Intervals:
  P1       :   0 - 4
  P4       :   4 - 5
  P2       :   5 - 8
  P3       :   8 - 11
  CPU IDLE :  11 - 20
  P5       :  20 - 22

 PID   AT   BT   CT   TAT   WT
---------------------------------
 P1   0    4    4    4     0
 P2   1    3    8    7     4
 P3   2    3    11   9     6
 P4   3    1    5    2     1
 P5   20   2    22   2     0
---------------------------------
Average Turnaround Time : 4.80
Average Waiting Time    : 2.20
```

Run 2: CPU idle at the start, equal burst and arrival time

Ubuntu Terminal

```
FCFS and SJF (Non-Preemptive) Scheduling

Enter number of processes: 3
--- Process 1 ---
  Process ID   : A
  Arrival Time : 2
  Burst Time   : 4
--- Process 2 ---
  Process ID   : B
  Arrival Time : 2
  Burst Time   : 4
--- Process 3 ---
  Process ID   : C
  Arrival Time : 3
  Burst Time   : 2

Original Process Data (same dataset for both algorithms)
 PID   AT   BT
 A    2    4
 B    2    4
 C    3    2

============================================================
  FCFS SCHEDULING
============================================================
Execution Sequence : IDLE -> A -> B -> C

Start/End Intervals:
  CPU IDLE :   0 - 2
  A        :   2 - 6
  B        :   6 - 10
  C        :  10 - 12

 PID   AT   BT   CT   TAT   WT
--------------------------------
 A    2    4    6    4     0
 B    2    4    10   8     4
 C    3    2    12   9     7
--------------------------------
Average Turnaround Time : 7.00
Average Waiting Time    : 3.67

============================================================
  SJF (NON-PREEMPTIVE) SCHEDULING
============================================================
Execution Sequence : IDLE -> A -> C -> B

Start/End Intervals:
  CPU IDLE :   0 - 2
  A        :   2 - 6
  C        :   6 - 8
  B        :   8 - 12

 PID   AT   BT   CT   TAT   WT
--------------------------------
 A    2    4    6    4     0
 B    2    4    12   10    6
 C    3    2    8    5     3
--------------------------------
Average Turnaround Time : 6.33
Average Waiting Time    : 3.00
```

Processes A and B have the same arrival and burst time, so A runs first because it was entered first. The CPU is idle from 0 to 2 because the first process arrives at time 2.

4 Result Comparison (Run 1 dataset)

Algorithm	Execution Sequence	Avg TAT	Avg WT
FCFS	P1 - P2 - P3 - P4 - IDLE - P5	5.60	3.00
SJF	P1 - P4 - P2 - P3 - IDLE - P5	4.80	2.20

SJF gives a lower average waiting time because the short process P4 runs before P2 and P3. Both algorithms show the same idle interval (11 - 20) because P5 arrives only at time 20.

5 Conclusion

FCFS and non-preemptive SJF were implemented in Python using the same process data. The program validates the input, keeps the original data unchanged, applies a clear tie-breaking rule, shows CPU idle intervals, and prints the sequence and start/end times for both algorithms. SJF gave a smaller average waiting and turnaround time than FCFS, while FCFS is simpler and follows the arrival order.